School of Engineering and Computing

MSc in Cyber Security

Interim Report

Project Title:

Explainable Dynamic Graph Neural Network SIEM for Software-Defined IoT using Edge AI and Federated Learning

by

Name: Arka Talukder

Date of submission: 23 March, 2026

Supervisor: Dr. Raja Ujjan

Moderator: Muhsin Hassanu

UWS UNIVERSITY OF THE WEST of SCOTLAND

DECLARATION OF ORIGINALITY

I declare that this is an original study based on my own work and that I have not submitted it for any other course or degree.

Signature_____ Anka _____

Table of Contents

Introduction 5

List of Figures 4

List of Tables 4

1. Summary Literature Review 5

1.1 IoT Security Landscape and Software-Defined IoT 6

1.2 Machine Learning for Network Intrusion Detection 7

1.3 Graph Neural Networks and Dynamic Graphs 7

1.4 Federated Learning for Privacy-Preserving IoT Security 8

1.5 Explainable AI for Security Operations 9

1.6 Research Gap and Contribution 10

2. Research Methodology 13

2.1 Research Approach and Basis 13

2.2 Dataset and Preprocessing 14

2.3 Graph Construction and Design Decisions 15

2.4 Model Architectures 15

2.5 Data Gathering and Analysis 16

2.6 Evaluation Metrics 16

2.7 Explainability and SIEM Integration 17

2.8 Academic Worth and Verifiable Goal 18

2.9 Anticipated Critical Reflection 18

3. Plan for Completion 19

3.1 Current Status 19

3.2 Plan to Completion 20

3.2.1 Alignment with Final Dissertation Criteria 21

3.3 How Results Will Be Analysed 22

3.4 Contingency Plans 23

References 24

List of Figures

1. **Figure 1:** Taxonomy of IDS approaches relevant to IoT and this project
2. **Figure 2:** Conceptual flow of a dynamic GNN (GAT + GRU)
3. **Figure 3:** Federated learning (FedAvg) flow
4. **Figure 4:** Explainability pipeline for SOC alerts
5. **Figure 5:** Positioning of related work (GNN, FL, explainability, SIEM)
6. **Figure 6:** Project integration of four literature pillars
7. **Figure 7:** End-to-end research pipeline aligned with literature findings
8. **Figure 8:** Interim-to-final completion roadmap

List of Tables

9. **Table 1:** Comparison of related work against project pillars
10. **Table 2:** Summary of model architectures
11. **Table 3:** Completion timeline
12. **Table 4:** Alignment of project progress with final dissertation criteria

Introduction

This interim report is submitted at the halfway point of my MSc project. It gives my supervisor and moderator a clear overview of the whole project so they can assess my progress and give feedback. This is not a first draft of the final report. It has three parts: (1) a summary literature review, (2) a description of my research methodology, and (3) a plan for completion.

My project aims to design and build a prototype system that can detect attacks in IoT network traffic. The system uses a dynamic graph neural network (GNN) with a Graph Attention Network (GAT) and a Gated Recurrent Unit (GRU). It is trained with federated learning so that raw data does not need to leave client sites. It also produces explainable alerts in a format that Security Operations Centre (SOC) analysts can use with their SIEM tools. The main research question is:

How can an explainable dynamic graph neural network, trained using federated learning, detect attacks in Software-Defined IoT flow data and generate SIEM alerts that support SOC operations on CPU-based edge devices?

At this point in the project, I have completed the data pipeline, built all four models (Random Forest, MLP, centralised GNN, and federated GNN), and set up the explainability and SIEM alert components. I have collected preliminary metrics but I have not yet done the final evaluation. The final results will be gathered, analysed and presented in the final dissertation report.

1. Summary Literature Review

This section summarises the key academic literature in my subject area and provides the framework for my research. I look at the strengths and limitations of prior work and identify the gap my project addresses. Figure 1 shows the main categories of intrusion detection relevant to IoT.

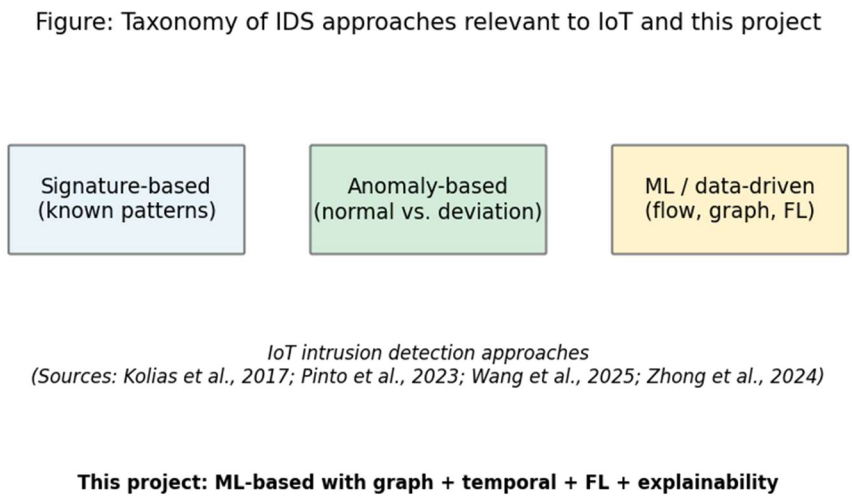

Figure 1: Taxonomy of IDS approaches relevant to IoT and this project.

Sources: Kolias et al. (2017); Pinto Neto et al. (2023); Wang et al. (2025); Zhong et al. (2024).

1.1 IoT Security Landscape and Software-Defined IoT

IoT devices are growing fast. This has created many security problems in consumer, industrial and critical infrastructure (Kolias et al., 2017). Many IoT devices use default passwords and have weak security. They also get few firmware updates. This makes them easy targets for attackers. When hacked, these devices can join large botnets like Mirai, which launched DDoS attacks over 1 Tbps (Antonakakis et al., 2017). The variety and number of IoT devices make defence hard. Old security models cannot monitor thousands of different devices easily.

Software-Defined IoT (SDIoT) helps with this problem. It separates the control plane from the data plane. This allows centralised network management through SDN controllers (Bera, Misra and Vasilakos, 2017). In SDIoT, the controller can see all forwarding decisions. It can also export flow data such as packet counts, byte counts and protocol breakdowns. This makes SDIoT a good fit for machine learning intrusion detection because the controller collects data from the whole network without needing software on each device. My project uses flow-level features from the CICIoT2023 dataset (Pinto Neto et al., 2023). These features are similar to what an SDN controller would produce.

However, Bera, Misra and Vasilakos (2017) mainly looked at SDN architecture. They did not show ML-based detection in that setup. Yang et al. (2025) recently showed that GNNs can improve intrusion detection for industrial IoT by modelling network structure, but they did not

combine this with federated learning or SIEM output. So there is still a gap between the SDIoT idea and real intrusion detection systems that use graphs, federated learning and explainability together. My project fills this gap.

1.2 Machine Learning for Network Intrusion Detection

Machine learning is now widely used for network intrusion detection. Models are trained on flow features to classify traffic as benign or malicious. Random Forest works well on network datasets. It does not overfit easily and it selects features through ensemble averaging (Ahmad et al., 2021). On CICIoT2023, Pinto Neto et al. (2023) found Random Forest got over 99% accuracy for binary detection. So I use it as a strong baseline in my project.

Deep learning approaches such as MLPs can learn complex patterns without manual feature engineering (Shone et al., 2018). Shone et al. (2018) used deep autoencoders with classifiers on NSL-KDD and got good results. However, NSL-KDD is smaller and less diverse than CICIoT2023. They also did not consider temporal relationships or links between flows. Both Random Forest and MLP treat each flow independently. They ignore connections between flows. This is a limitation because, in real networks, flows are linked. For example, a scan often comes before an attack. Command-and-control traffic connects to data theft. If we ignore these links, we miss multi-stage attacks (Caville et al., 2022). Wang et al. (2025) did a scoping review and found that graph-based approaches to network traffic analysis are used more and more. Zhong et al. (2024) surveyed GNNs for intrusion detection and noted that explainability and federated deployment are still under-explored. These findings support the graph-based approach in this project.

1.3 Graph Neural Networks and Dynamic Graphs

Graph Neural Networks (GNNs) model how network entities relate to each other. Nodes can represent flows or hosts, and edges show communication links. The GNN learns from its neighbours through message passing (Caville et al., 2022). Graph Attention Networks (GATs) go further. They learn to give different weights to different neighbours, so the model focuses on the most important connections (Velickovic et al., 2018). This helps in two ways: it improves classification and it shows which flows influenced each decision.

Basak et al. (2025) used GNNs for intrusion detection on UNSW-NB15. They built graphs from flow features with edges linking similar flows. They got F1 over 97%. However, a key limitation is that they used static graphs. The whole dataset was one snapshot. They did not capture how behaviour changes over time. Also, UNSW-NB15 is older and has fewer attack types than

CICIoT2023. Han et al. (2025) proposed DIMK-GCN for intrusion detection and showed that multi-scale graph representations can capture both local and global attack patterns. Ngo et al. (2025) used GNNs on IoT datasets with kNN-based graph construction. Edges linked flows that were similar in feature space. This worked without needing device topology. They found k=5 is a good default. However, they also used static graphs with no temporal modelling.

My project builds on these studies. I add a GRU layer that processes sequences of graph snapshots. So the system can learn from how the graph changes over time, not just from single snapshots. Figure 2 shows the conceptual flow of this dynamic GNN architecture.

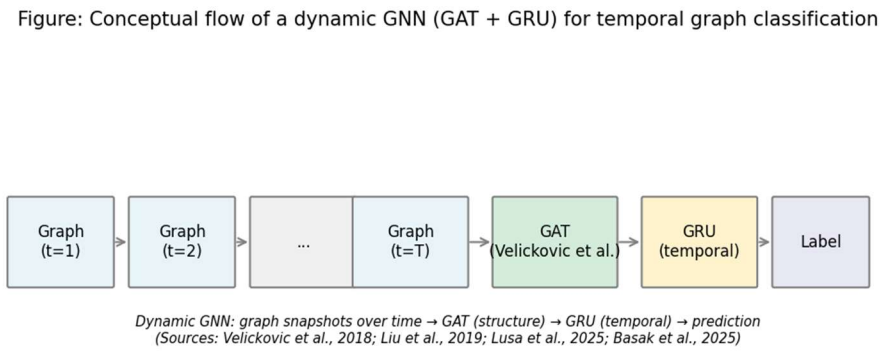

Figure 2: Conceptual flow of a dynamic GNN (GAT + GRU) for temporal graph classification.

Sources: Velickovic et al. (2018); Liu et al. (2019); Basak et al. (2025).

1.4 Federated Learning for Privacy-Preserving IoT Security

Federated learning (FL) lets many clients train a model together without sharing raw data. This helps with privacy and regulations like GDPR (McMahan et al., 2017). In FedAvg, each client trains on its own data and only sends model updates to a server. The server averages these updates and sends the improved model back to all clients. This fits IoT security well because traffic data often comes from different organisations or places and cannot be pooled in one location. Figure 3 shows the FedAvg flow used in this project.

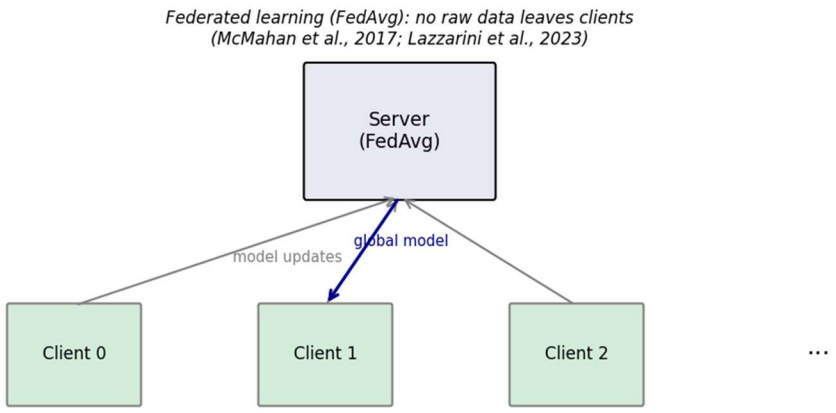

Figure 3: Federated learning (FedAvg) flow — no raw data leaves clients.

Sources: McMahan et al. (2017); Lazzarini et al. (2023).

Lazzarini et al. (2023) used federated learning for intrusion detection on UNSW-NB15. Their federated MLP got within 2% F1 of the centralised model. However, they only used two clients with IID (evenly distributed) data. This is an ideal case that does not reflect real IoT deployments. In practice, different networks (for example, factory sensors compared to smart homes) see very different traffic patterns. This non-IID data is known to hurt FedAvg performance (Kairouz et al., 2021).

Albanbay et al. (2025) addressed non-IID data by using Dirichlet splits. Lower alpha values (meaning more different data across clients) caused up to 8% F1 drop. It also needed about twice as many rounds to converge. However, they used flat models, not graph models. It is still unclear whether graph-based models behave differently with non-IID data. My project tests this by combining GNNs with federated learning using alpha=0.5.

1.5 Explainable AI for Security Operations

SOCs receive too many alerts. Analysts handle thousands each day. Many alerts are false positives or lack context (Alahmadi et al., 2022). Cuppens and Miege (2002) proposed alert correlation to reduce noise, but the main problem remains: many detection systems output labels without saying why. ML models that only give a yes or no answer make this problem worse.

Explainable AI (XAI) helps by showing which input features drove each decision. Integrated Gradients (Sundararajan, Taly and Yan, 2017) does this by following a path from a baseline to the real input. It has good mathematical properties. Captum (Kokhlikyan et al., 2020) implements Integrated Gradients for PyTorch. I use Captum in my project. Alabbadi and Bajaber (2025) used

SHAP for IoT intrusion detection. Feature attribution helped analysts trust the alerts. However, SHAP is slow for deep models. One prediction can take over 10 seconds. That is too slow for real-time or edge use. Integrated Gradients is faster because it needs only one forward and backward pass.

GAT attention weights provide another form of explanation. They show which neighbour flows influenced each classification. No prior work I found combines both approaches: feature attributions from Integrated Gradients and structural explanations from attention weights. I use both in my project to give analysts richer context for each alert. Figure 4 shows the explainability pipeline used in this project.

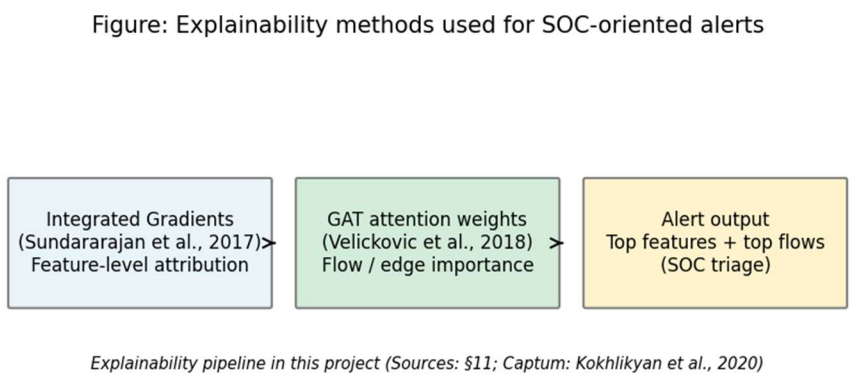

Figure 4: Explainability pipeline — Integrated Gradients and GAT attention for SOC alerts.

Sources: Sundararajan et al. (2017); Velickovic et al. (2018); Kokhlikyan et al. (2020).

1.6 Research Gap and Contribution

Table 1 compares the main related works against the four pillars of my project. The table shows that each study covers one or two areas, but no single study covers all four pillars together.

Table 1: Comparison of Related Work Against Project Pillars

Study	Dynamic GNN	Federated Learning	Explainability	SIEM Integration
Basak et al. (2025)	Static GNN	No	Partial	No
Ngo et al. (2025)	Static GNN	No	No	No
Han et al. (2025)	Static GNN	No	No	No
Yang et al. (2025)	Yes	No	No	No

Lazzarini et al. (2023)	No	Yes (IID only)	No	No
Albanbay et al. (2025)	No	Yes (non-IID)	No	No
Alabbadi and Bajaber (2025)	No	No	Yes (SHAP)	No
This project	**Yes (GAT+GRU)**	**Yes (non-IID)**	**Yes (IG+Attention)**	**Yes (ECS JSON)**

No study I found combines all four of these in one system: dynamic GNN with temporal modelling, federated learning with non-IID data, dual explainability (feature-level and structural), and SIEM-style alerts. Figure 5 shows the positioning of related work against these four pillars. My project fills this gap. I am building a system that has all four components and runs on CPU for edge deployment, using the CICIoT2023 dataset.

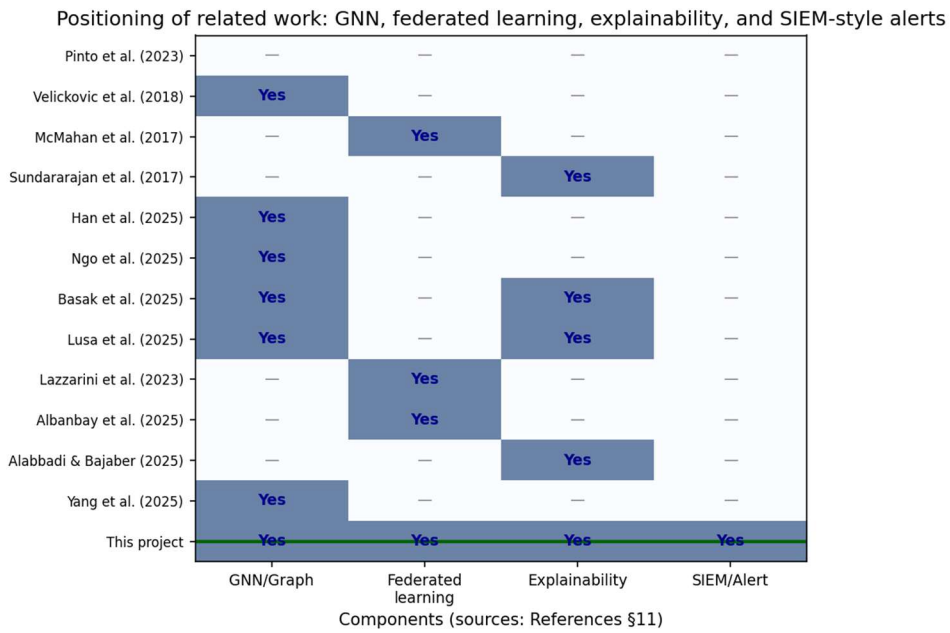

Figure 5: Positioning of related work — GNN, federated learning, explainability, and SIEM.

Sources: All references cited in Table 1. This figure synthesises the literature review.

To make the project positioning clearer, Figure 6 maps the four literature pillars directly to the integrated design used in this project.

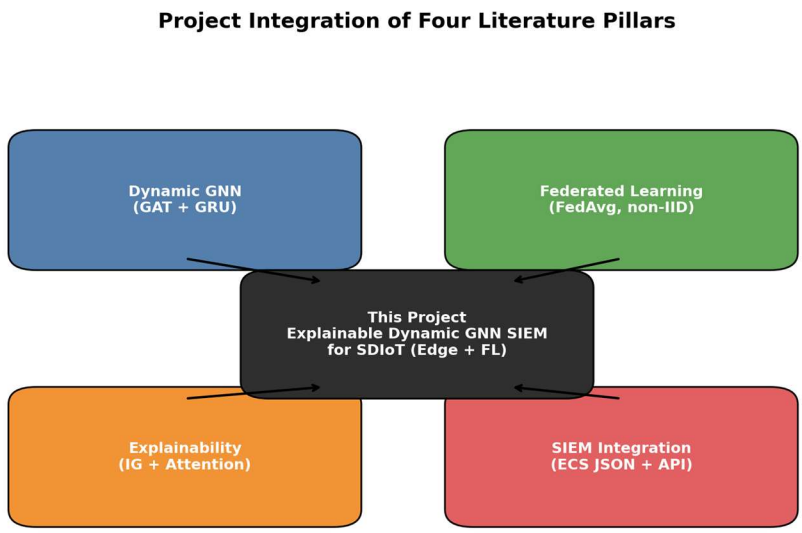

Figure 6: Project integration of four literature pillars.

Finally, Figure 7 presents the end-to-end project pipeline, included here to show how the literature insights are translated into an operational research design.

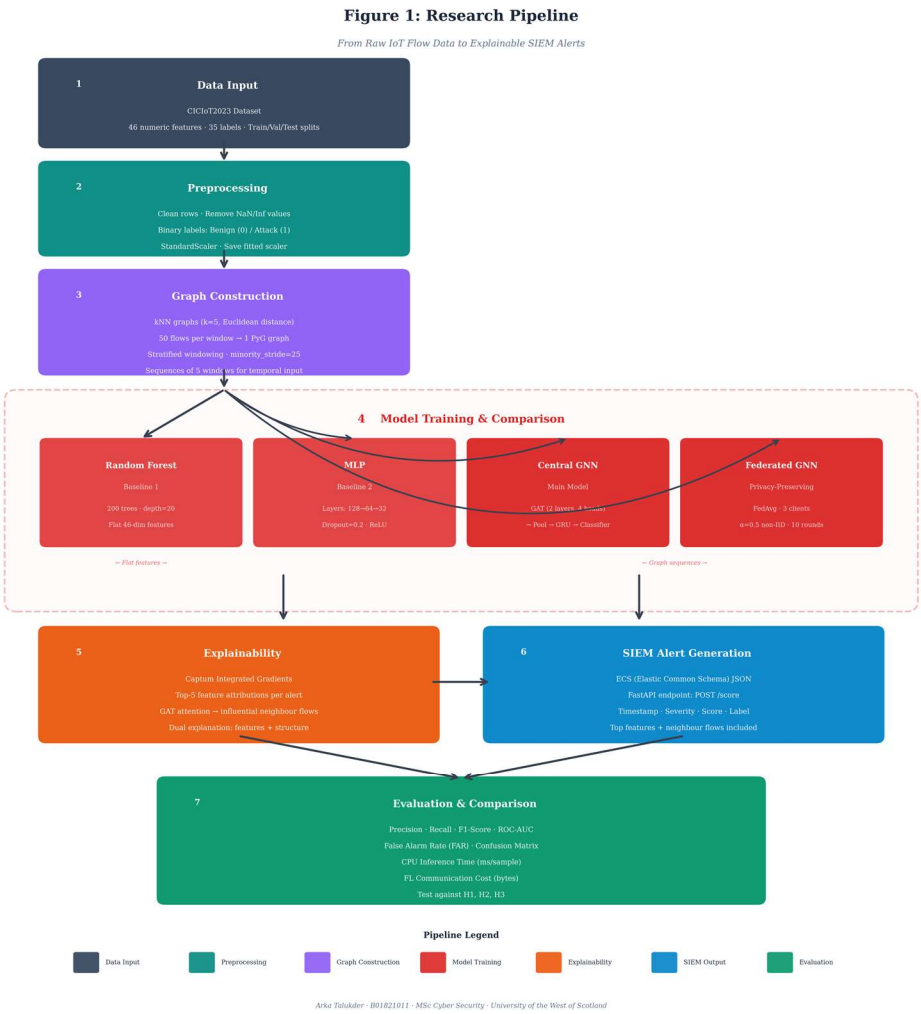

Figure 7: End-to-end research pipeline aligned with literature findings.

2. Research Methodology

This section explains how I am doing my research. It covers my method, how I collect and analyse data, and the academic goal of the project.

2.1 Research Approach and Basis

My approach is quantitative and experimental. I design, build and compare several detection models under controlled conditions. I measure metrics like precision, recall, F1, ROC-AUC, false

alarm rate and CPU inference time. The test set is held out and never used during training. Each model is tested against the hypotheses below.

Primary research question: How can an explainable dynamic graph neural network, trained using federated learning, detect attacks in Software-Defined IoT flow data and generate SIEM alerts that support SOC operations on CPU-based edge devices?

Sub-questions and hypotheses:

- **H1:** The dynamic GNN (GAT+GRU) achieves a higher F1 score and lower false alarm rate than Random Forest and MLP baselines on CICIoT2023 binary classification.
- **H2:** The federated GNN achieves F1 within 2% of the centralised GNN, showing that privacy-preserving training does not substantially degrade detection performance.
- **H3:** Integrated Gradients and GAT attention weights produce feature attributions and flow-level explanations that identify attack-relevant features (such as TCP flags and protocol indicators) in the generated SIEM alerts.

2.2 Dataset and Preprocessing

I use the CICIoT2023 dataset (Pinto Neto et al., 2023). It has flow-level records with 46 numeric features including packet counts, byte counts, flow duration, protocol indicators (TCP, UDP, ICMP, HTTP, DNS), TCP flag counts (SYN, ACK, RST, PSH, FIN), and statistical measures (mean, variance, standard deviation). Labels cover benign traffic and 34 attack types including DDoS, DoS, reconnaissance, brute force and spoofing. A manageable subset of 500,000 flows per split is selected to fit the 15-week timeline.

Preprocessing steps:

13. Load raw CSVs and remove any rows with missing or infinite values.
14. Map the 35 original labels to binary classes: 0 (Benign) and 1 (Attack).
15. Apply StandardScaler (fitted on training data only) to normalise all 46 features to zero mean and unit variance. Save the fitted scaler so the same transformation is applied to validation and test sets without data leakage.
16. Export normalised data as Parquet files for efficient loading in later steps.

I use binary classification to keep the scope manageable within the 15-week project. The main SOC question is: is this traffic malicious or not? Multi-class classification can be added in future work.

2.3 Graph Construction and Design Decisions

The public version of CICIoT2023 has no device IDs or IP addresses. So I cannot build a device-topology graph. Instead, I follow the approach of Ngo et al. (2025) and Basak et al. (2025) and build graphs from feature similarity:

- **Nodes:** Each flow record becomes a node with its 46 normalised features as the node feature vector.
- **Edges:** k-nearest neighbours (kNN) in Euclidean feature space, with k=5. Ngo et al. (2025) validated this value as a good balance between graph connectivity and computational cost.
- **Windows:** 50 flows per window. This gives enough nodes for the GAT to learn meaningful patterns, while keeping computation fast enough for CPU inference.
- **Sequences:** 5 consecutive windows per sample. These are fed to the GRU to capture how network behaviour changes over time (covering 250 flows in total).
- **Stratified windowing:** The dataset is heavily imbalanced (approximately 97.8% attack, 2.2% benign at flow level). To handle this, benign and attack flows are separated into pools. Windows are built from each pool independently. Minority-class windows use an overlapping stride (25 flows) for augmentation. Sequences are then balanced and shuffled before training.

2.4 Model Architectures

I train and compare four models. They represent increasing structural sophistication. Table 2 summarises the key parameters of each model.

Table 2: Summary of Model Architectures

Model	Type	Input	Key Parameters	Graph/Temporal
Random Forest	Ensemble	46 flow features	200 trees, max depth 20	No / No
MLP	Neural network	46 flow features	3 layers (128, 64, 32), dropout 0.2	No / No
Centralised GNN	GAT + GRU	kNN graph sequences	4 heads, dim 64, GRU dim 64	Yes / Yes
Federated GNN	GAT + GRU	kNN graph sequences	Same as above, 3 clients, 10 rounds	Yes / Yes

- **Random Forest (Baseline 1):** 200 decision trees with maximum depth 20. Trained on flat 46-dimensional feature vectors. This is a strong non-neural baseline that is fast and resistant to overfitting.
- **MLP (Baseline 2):** Three hidden layers (128, 64, 32 neurons) with ReLU activation and 20% dropout. Trained on the same flat features. This provides a neural baseline without graph or temporal components.
- **Centralised Dynamic GNN (Main Model):** This is the core model. It has: (1) two GAT convolutional layers (4 attention heads, hidden dimension 64); (2) global mean pooling to get one vector per graph snapshot; (3) a GRU recurrent layer (hidden dimension 64) that processes the sequence of 5 graph embeddings; (4) a fully connected classifier that produces binary attack/benign probability.
- **Federated Dynamic GNN:** The same architecture as the centralised GNN, but trained using FedAvg across 3 simulated clients. Data is split using Dirichlet distribution (alpha=0.5) to create non-IID conditions. Training runs for 10 communication rounds with 2 local epochs per round. Flower (Beutel et al., 2020) is used as the federated learning framework.

2.5 Data Gathering and Analysis

I use the pre-defined train/validation/test splits from CICIoT2023. This lets me compare my results with other papers that use the same dataset. I use the validation set for hyperparameter tuning and early stopping (patience=5 epochs). The test set is reserved only for final evaluation. A fixed random seed (42) makes results reproducible. I use class-weighted loss for all neural models. This prevents the model from simply predicting the majority class. One challenge I faced during data preparation was handling the extreme class imbalance at flow level. The raw data had about 97.8% attack flows and only 2.2% benign flows. If I trained on this directly, the model would just predict attack every time. I solved this by building the stratified windowing strategy described in Section 2.3, which balances the graph-level samples before training.

2.6 Evaluation Metrics

I use these metrics because they matter for SOC work.

Precision (Eq. 1): the fraction of predicted attacks that are actually attacks.

$$Precision = TP / (TP + FP) \tag{1}$$

Recall (Eq. 2): the fraction of actual attacks that the model correctly finds.

$$Recall = TP / (TP + FN) \tag{2}$$

F1-score (Eq. 3): the harmonic mean of precision and recall.

$$F_1 = 2 \times Precision \times Recall \,/\, (Precision + Recall) \tag{3}$$

False Alarm Rate (Eq. 4): the fraction of benign traffic incorrectly flagged as an attack.

$$FAR = FP \,/\, (FP + TN) \tag{4}$$

ROC-AUC (Eq. 5): area under the Receiver Operating Characteristic curve. The ROC curve plots true positive rate against false positive rate at varying thresholds. AUC of 1.0 indicates perfect ranking; 0.5 indicates random guessing.

$$AUC = \int_0^1 TPR(t)\, d(FPR(t)) \tag{5}$$

I also use a confusion matrix, CPU inference time (ms per sample), and communication cost (bytes sent during federated training). False alarm rate matters most for SOCs because high recall with high FAR means too many false alerts.

2.7 Explainability and SIEM Integration

I use two methods for explainability:

17. **Integrated Gradients (Captum):** For each flagged flow, feature attributions are computed by accumulating gradients along a path from a zero-valued baseline to the actual input (Eq. 6). The top-5 contributing features are extracted and included in the alert.

$$IG_i(x) = (x_i - x'_i) \times \int_0^1 \partial F(x' + \alpha(x - x')) \,/\, \partial x_i \, d\alpha \tag{6}$$

where x is the input, x′ is the baseline, and F is the model output.

18. **GAT attention weights:** The learned attention coefficients from the final GAT layer identify which neighbouring flows in the graph most influenced the classification. This provides structural context.

Alerts are formatted in ECS (Elastic Common Schema) JSON format. I serve them through a FastAPI endpoint (POST /score). Each alert includes: timestamp, severity level, confidence score, predicted label, top contributing features, and top neighbour flows. This format works with SIEM platforms like Elastic Security and Splunk.

The full research pipeline from raw data to SIEM alerts is shown earlier in Figure 7 and is used here as the implementation reference for this section. During implementation, I found that running Integrated Gradients on graph sequences was slower than expected because Captum needs many forward passes (50 integration steps by default). To keep inference time acceptable on CPU, I set the system to compute explanations only for flows flagged as attacks, rather than

for every single flow. This reduced the explainability overhead without losing the most important alerts.

2.8 Academic Worth and Verifiable Goal

My project has a clear, verifiable academic goal: to test whether an explainable dynamic GNN, trained with federated learning, can detect IoT attacks and produce useful SIEM alerts on CPU-based edge devices. I will demonstrate the rigour of my research in the following ways:

19. **Architectural contribution:** No existing prototype combines dynamic GNN (GAT+GRU), federated learning, dual explainability, and SIEM integration in one system on CICIoT2023 (see Table 1).
20. **Empirical evidence:** I compare flat models (Random Forest, MLP), centralised GNN, and federated GNN on the same data with identical preprocessing. This provides objective evidence of whether graph structure and federated learning actually improve detection.
21. **Practical applicability:** The full pipeline runs on CPU with acceptable latency for edge use. This bridges the gap between academic research and real SOC tools.

How I will present the data to prove rigour: I will present results in comparison tables, confusion matrices, ROC curves, and federated learning convergence plots. Each hypothesis (H1 to H3) maps to specific metrics that can be checked on the held-out test set. I will not use the test set for any tuning. It is reserved solely for final evaluation. Beyond the performance numbers, I believe the project is useful because it shows that all four components (graph detection, federated training, explainability, and SIEM output) can work together in one small system on a normal computer. Most papers only look at one or two of these parts. Showing that they can be combined in a working prototype is valuable for both researchers and for SOC teams who want to try graph-based detection without needing expensive hardware.

2.9 Anticipated Critical Reflection

The MSc Project Handbook requires a Critical Reflection section in the final report. At this stage, I anticipate reflecting on the following areas:

- **Dataset representativeness:** CICIoT2023 is a controlled testbed dataset. Results may not directly transfer to all real IoT environments with different devices and traffic patterns.
- **Graph construction validity:** kNN feature-similarity graphs are a proxy for true network topology. The assumption that flows similar in feature space are behaviourally related may not always hold, especially for encrypted traffic.

- **Federated learning scale:** Three simulated clients on a single machine do not capture real-world communication latency, client dropout, or security attacks on the federation itself.
- **SDIoT scope:** The flow features are consistent with SDN controller exports. However, the prototype does not integrate with an actual SDN controller. The SDIoT framing provides architectural context rather than a demonstrated deployment.
- **Explainability validation:** Without a formal user study with SOC analysts, the claim that explanations "support" triage is based on qualitative assessment rather than measured analyst performance improvement.
- **Prototype maturity:** The system is a research prototype. It is not a production SIEM.

These limitations will define clear boundaries for my conclusions and identify directions for future work. I will incorporate feedback from this interim report into the critical reflection section of my final report. If I were to restart this project, I would spend more time early on exploring the dataset before jumping into graph construction. I spent several days debugging the kNN graph builder before I realised that some features had very different scales, which made the distance calculations unreliable until I applied proper normalisation. Starting with a thorough data exploration step would have saved that time.

3. Plan for Completion

This section shows where the project is now and how I plan to finish it. It also covers what I will do if results are not as expected.

3.1 Current Status

At the time of writing this report, I have completed the following:

- **Data pipeline:** CICIoT2023 has been downloaded, preprocessed, normalised, and exported as Parquet files. The fitted scaler has been saved for consistent transformation of validation and test sets.
- **Graph construction:** The kNN graph builder, stratified windowing, and sequence dataset have been implemented and tested.
- **All four models:** Random Forest, MLP, centralised GNN, and federated GNN architectures have been implemented and trained.
- **Training infrastructure:** Training loops with early stopping, class-weighted loss, and metric logging are operational.

- **Federated learning:** Flower server and client scripts are implemented. Dirichlet non-IID splitting is functional. A 10-round simulation has been completed.
- **Explainability:** Captum Integrated Gradients and GAT attention extraction are implemented and produce feature attributions.
- **SIEM integration:** The FastAPI endpoint producing ECS-formatted JSON alerts is operational.
- **Preliminary metrics:** All four models have been trained and evaluated on the test set. Initial metrics have been collected but not yet formally analysed.

The main engineering work is done. What remains is final evaluation, analysis, and writing the dissertation.

3.2 Plan to Completion

Table 3: Completion Timeline

Week	Dates (Approx.)	Tasks	Deliverables
5	Current week	Finalise all experiments with fixed seeds; generate final metrics for all four models	Final metric tables, confusion matrices
5–6	+1 week	Generate all figures: confusion matrices, ROC curves, FL convergence plot, inference time chart	All dissertation figures
6	+1.5 weeks	Conduct sensitivity analysis: vary k (3, 5, 7) and window_size (25, 50, 100) if time permits	Sensitivity analysis table (optional)
6–7	+2 weeks	Write Results and Discussion chapters; interpret results against H1 to H3	Draft Results and Discussion
7	+2.5 weeks	Write Conclusion and Critical Self-	Draft Conclusion chapter

		Evaluation; revisit references	
7–8	+3 weeks	Incorporate interim report feedback; integrate all chapters; proofread	Complete dissertation draft
8	Submission week	Final proofread; check word count; verify figures; submit	Final dissertation submission

Figure 8 provides a visual roadmap of the remaining project phases from interim submission to final submission.

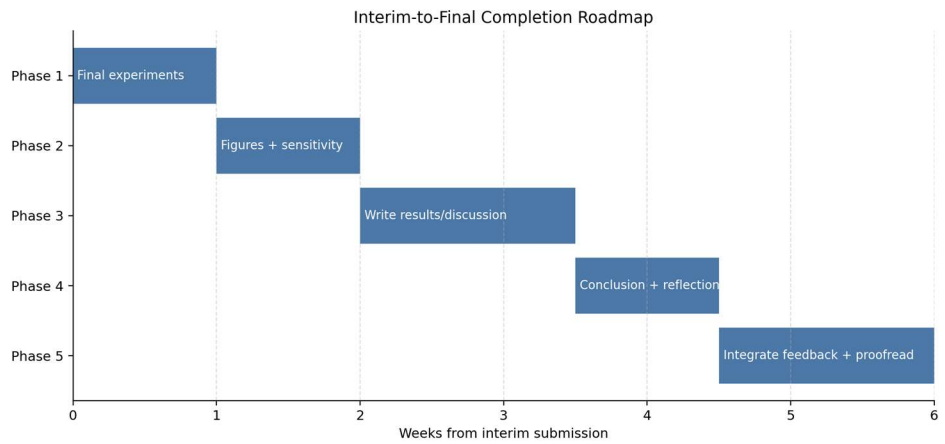

Figure 8: Interim-to-final completion roadmap.

3.2.1 Alignment with Final Dissertation Criteria

Table 4 shows how my progress aligns with the final dissertation marking criteria.

Table 4: Alignment of Project Progress with Final Dissertation Criteria

Final Criterion	Weight	Interim Status	Planned Improvement Before Final Submission
Introduction	5%	Drafted clearly in final dissertation structure	Tighten motivation and scope after final results
Context / Literature Review	20%	Strong and expanded with critical analysis,	Add any new references prompted by final

		figures, and gap table	findings
Research Design	20%	Methodology fully specified with equations, hypotheses, and controls	Add final validation details and any sensitivity outcomes
Implementation (Practical Work)	25%	Core system completed (data pipeline, 4 models, FL, XAI, API)	Improve code documentation and reproducibility notes
Evaluation	5%	Preliminary metrics collected	Run final fixed-seed experiments and verify consistency
Presentation of Results	5%	Result plan and figure templates prepared	Insert final tables/plots with concise interpretation
Conclusions and Recommendations	10%	Draft direction defined in completion plan	Finalise evidence-based recommendations from results
Critical Self-Evaluation	10%	Anticipated reflection areas identified	Write full reflective analysis after final evaluation

3.3 How Results Will Be Analysed

When the final results are ready, I will analyse them as follows:

22. **Model comparison (H1):** I will compare Random Forest, MLP, centralised GNN, and federated GNN on the same test set using precision, recall, F1, ROC-AUC, and false alarm rate. Results will be presented in a comparison table.
23. **Federated vs. centralised (H2):** I will compare the federated model's metrics round by round and check whether it reaches performance within 2% of the centralised model. Communication cost will also be reported.
24. **Explainability assessment (H3):** I will generate 3 to 5 example alerts with full explanations (top features and top flows). I will discuss whether the highlighted features would help a SOC analyst understand and act on each alert. This assessment is qualitative.

25. **Figures and visualisations:** Confusion matrices for each model, ROC curves, a federated convergence plot, and a bar chart comparing inference time and F1-score will support the text.

One thing I noticed in the preliminary runs is that the centralised GNN reached very high validation scores after just one or two epochs. This fast convergence might mean the chosen subset is relatively easy for the model, or it might mean the class-weighted loss and stratified windowing are working very well together. I plan to look into this more carefully during the final evaluation. If the task is too easy, I may try a harder subset or discuss it as a limitation.

3.4 Contingency Plans

I have planned for the possibility that results do not go as expected:

- **Scenario 1: GNN does not beat baselines.** That is still a valid result. I will discuss why flat models might be sufficient for this dataset. The contribution then becomes evidence about when graphs help and when they do not.
- **Scenario 2: Federated GNN drops a lot (more than 5% F1).** I will investigate whether the drop is caused by non-IID data or too few rounds. I may suggest alternatives like FedProx or more communication rounds for future work.
- **Scenario 3: Explainability is too slow.** I will apply it only to selected alerts (for example, the top 10% by confidence). I will document the trade-off and may suggest faster methods.
- **Scenario 4: Running out of time.** I will drop the sensitivity analysis first. The main deliverable (four models, explanations, and SIEM alerts) can still be completed without it.

References

1. Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J. and Ahmad, F. (2021) 'Network intrusion detection system: a systematic study of machine learning and deep learning approaches', *Transactions on Emerging Telecommunications Technologies*, 32(1), e4150. doi: 10.1002/ett.4150.
2. Alahmadi, B.A., Axon, L., Martinovic, I. and Sherr, M. (2022) '99% false positives: a qualitative study of SOC analysts' perspectives on security alarms', in *Proceedings of the 31st USENIX Security Symposium*. Boston, MA, 10–12 August. Berkeley, CA: USENIX Association, pp. 2783–2800.
3. Alabbadi, A. and Bajaber, F. (2025) 'An intrusion detection system over the IoT data streams using eXplainable artificial intelligence (XAI)', *Sensors*, 25(3), p. 847. doi: 10.3390/s25030847.
4. Albanbay, N., Tursynbek, Y., Graffi, K., Uskenbayeva, R., Kalpeyeva, Z., Abilkaiyr, Z. and Ayapov, Y. (2025) 'Federated learning-based intrusion detection in IoT networks: performance evaluation and data scaling study', *Journal of Sensor and Actuator Networks*, 14(4), p. 78. doi: 10.3390/jsan14040078.
5. Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J.A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K. and Zhou, Y. (2017) 'Understanding the Mirai botnet', in *Proceedings of the 26th USENIX Security Symposium*. Vancouver, BC, 16–18 August. Berkeley, CA: USENIX Association, pp. 1093–1110.
6. Basak, M., Kim, D.-W., Han, M.-M. and Shin, G.-Y. (2025) 'X-GANet: an explainable graph-based framework for robust network intrusion detection', *Applied Sciences*, 15(9), p. 5002. doi: 10.3390/app15095002.
7. Bera, S., Misra, S. and Vasilakos, A.V. (2017) 'Software-defined networking for Internet of Things: a survey', *IEEE Internet of Things Journal*, 4(6), pp. 1994–2008. doi: 10.1109/JIOT.2017.2746186.
8. Beutel, D.J., Tober, T., Steiner, D., Aber, S., Becker, T., Naseri, A., Tanner, Y. and Lane, N.D. (2020) 'Flower: a friendly federated learning framework', *arXiv preprint*. doi: 10.48550/arXiv.2007.14390.
9. Caville, E., Lo, W.W., Layeghy, S. and Portmann, M. (2022) 'Anomal-E: a self-supervised network intrusion detection system based on graph neural networks', *Knowledge-Based Systems*, 258, p. 110030. doi: 10.1016/j.knosys.2022.110030.

10. Cuppens, F. and Miege, A. (2002) 'Alert correlation in a cooperative intrusion detection framework', in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pp. 202–215.
11. Han, Z., Zhang, C., Yang, G., Yang, P., Ren, J. and Liu, L. (2025) 'DIMK-GCN: a dynamic interactive multi-channel graph convolutional network model for intrusion detection', *Electronics*, 14(7), p. 1391. doi: 10.3390/electronics14071391.
12. Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R. et al. (2021) 'Advances and open problems in federated learning', *Foundations and Trends in Machine Learning*, 14(1–2), pp. 1–210. doi: 10.1561/2200000083.
13. Kokhlikyan, N., Miglani, V., Martin, M., Wang, E., Alsallakh, B., Reynolds, J., Melnikov, A., Kliushkina, N., Araya, C., Yan, S. and Reblitz-Richardson, O. (2020) 'Captum: a unified and generic model interpretability library for PyTorch', *arXiv preprint*. doi: 10.48550/arXiv.2009.07896.
14. Kolias, C., Kambourakis, G., Stavrou, A. and Voas, J. (2017) 'DDoS in the IoT: Mirai and other botnets', *Computer*, 50(7), pp. 80–84. doi: 10.1109/MC.2017.201.
15. Lazzarini, R., Tianfield, H. and Charissis, V. (2023) 'Federated learning for IoT intrusion detection', *AI*, 4(3), pp. 509–530. doi: 10.3390/ai4030027.
16. Liu, Y., Chen, J. and Zhou, J. (2019) 'Temporal graph neural networks for fraud detection', in *Proceedings of the 2019 IEEE International Conference on Data Mining (ICDM)*, pp. 1202–1207.
17. McMahan, B., Moore, E., Ramage, D., Hampson, S. and Arcas, B.A. (2017) 'Communication-efficient learning of deep networks from decentralized data', in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Fort Lauderdale, FL, 20–22 April. PMLR, pp. 1273–1282.
18. Ngo, T., Yin, J., Ge, Y.-F. and Wang, H. (2025) 'Optimizing IoT intrusion detection: a graph neural network approach with attribute-based graph construction', *Information*, 16(6), p. 499. doi: 10.3390/info16060499.
19. Pinto Neto, E.C., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R. and Ghorbani, A.A. (2023) 'CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment', *Sensors*, 23(13), p. 5941. doi: 10.3390/s23135941.
20. Shone, N., Ngoc, T.N., Phai, V.D. and Shi, Q. (2018) 'A deep learning approach to network intrusion detection', *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1), pp. 41–50. doi: 10.1109/TETCI.2017.2772792.

21. Sundararajan, M., Taly, A. and Yan, Q. (2017) 'Axiomatic attribution for deep networks', in *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Sydney, Australia, 6–11 August. PMLR, pp. 3319–3328.
22. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P. and Bengio, Y. (2018) 'Graph attention networks', in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*. Vancouver, BC, 30 April – 3 May. Available at: https://openreview.net/forum?id=rJXMpikCZ.
23. Wang, R., Zhao, J., Zhang, H., He, L., Li, H. and Huang, M. (2025) 'Network traffic analysis based on graph neural networks: a scoping review', *Big Data and Cognitive Computing*, 9(11), p. 270. doi: 10.3390/bdcc9110270.
24. Yang, S., Pan, W., Li, M., Yin, M., Ren, H., Chang, Y., Liu, Y., Zhang, S. and Lou, F. (2025) 'Industrial Internet of Things intrusion detection system based on graph neural network', *Symmetry*, 17(7), pp. 997–997. doi: 10.3390/sym17070997.
25. Zhong, M., Lin, M., Zhang, C. and Xu, Z. (2024) 'A survey on graph neural networks for intrusion detection systems: methods, trends and challenges', *Computers and Security*, 141, pp. 103821. doi: 10.1016/j.cose.2024.103821.